Account Health: AI build log

Track 2

Prototype: razorpay-account-health-demo.lovable.app

I used AI tools for research, coding, writing and testing. The problem, the scope and the product decisions were mine. Below is what each tool did and what I decided at each stage.

How I used AI, stage by stage

Stage	Tool	What AI did	What I decided
Brief and first idea	Perplexity	Summarised the brief, compared the two tracks and Razorpay's agent products, and drafted a first idea on failed-payment recovery.	Picked Track 2. Dropped the first idea because it was generic and not based on real merchant complaints.
Problem discovery	Claude + Python	Wrote a script to scrape and tag Google Play reviews. I ran it and got 432 reviews of the Razorpay Payments app (Sep 2025 to Sep 2026).	Chose settlement holds and account suspensions as the problem. A recount showed 13.7% (32 of 234 one-star reviews), lower than my first estimate of 21.8%, so I used the lower figure.
Existing products	Claude (deep research)	Mapped Agent Studio, the Agentic Dashboard, RAY and Razorpay's settlement and dispute docs.	Positioned Account Health on top of existing agents like Dispute Responder instead of rebuilding them, and made clear how it differs from RAY.
Merchant interview	None (my own)	–	Spoke to one founder. They wanted to see the transactions behind any flag and to approve actions first. I added severity labels and a false-alerts metric.
Solution design	Claude	Helped structure the concept, the split between rules and AI, failure cases and metrics, and drafted the product note.	Kept the score rules-based and used AI only for the plan. Every step needs merchant approval. Thought through how a fraudster could misuse it: new merchants see bands, not exact limits, and AML or fraud holds are never explained.
Reviews	Claude, ChatGPT	Critiqued the spec and checked every number in the note against the prototype.	Took some suggestions: a released settlement is working capital, not revenue; the courier link is a pattern, not proof. Said no to a rename, made-up metric targets and alarming warning text.
Whiteboarding	Claude (design canvas)	Drew the three screens on a canvas so I could see the whole flow before building anything.	Set the design goals: answer what's wrong, why, and what to do, in that order; one main action per screen; evidence before advice. After seeing it standalone, I moved it inside the real dashboard menu and added the chargeback table.
Build	Claude, Lovable	Claude turned the whiteboard into a build prompt. Lovable built the app.	Kept it to three screens to fit Lovable's free credits. Used a screenshot of the real dashboard so the menu matches.
Testing	Claude + me	Claude clicked through every page and edge case in the live app.	I went through every click and page myself to check it was usable, intuitive and minimal. This led to a summary at the top of the hold page, clearer step states and simpler wording. I fixed the issues that affected the flow.

Where AI saved time

- **Evidence:** a tagged review dataset in under an hour instead of days of reading.
- **Research:** Razorpay's current products mapped in one sitting, which changed how I positioned the idea.
- **Prototype:** a working three-page app built and tested in a day without writing code.
- **Checking:** caught mistakes I had missed, including numbers that didn't match.

Where I went against AI

- Dropped the first concept and the 21.8% figure when the data didn't support them.
- Kept the name Account Health instead of a suggested rename.
- Left out made-up targets and policy claims; metrics show direction until there's a baseline.

All prototype data is synthetic. External tools: Google Play review data (public, collected with the open-source google-play-scraper Python library), Lovable, Claude, Perplexity, ChatGPT.